

Customer Segmentation: Clustering and PCA

MKT 566 · Fall 2026

Davide Proserpio

USC Marshall School of Business

Before We Start

Housekeeping

- **Airbnb case report** ([airbnb-report.html](#) + [ai-log.md](#)) due **Sunday, Sept. 27, 11:59 pm** on Brightspace. Tasks 6–8 are yours to finish
- **Homework 2** (box office) due **Thursday, Oct. 8**
- **Group project**: proposal presentations on **Oct. 13 and 15**
- Today: **clustering** (k-means) and **PCA**, with two scripts you run along with me. Tuesday: **recommendation systems**

A different kind of question

Every model so far had an **outcome** to predict: price on Thursday, gem on Tuesday. Today there is no outcome. There are just customers, described by numbers, and one question: **who is similar to whom?**

What changes

- No Y , no coefficients, no stars: the algorithm proposes **groups**
- This is called **unsupervised** learning: nothing tells the algorithm what the right answer is

What stays

- The data is still rows and columns, and **you** choose the columns
- The output still means nothing until **you** interpret it
- Still **not causation**: clusters describe, they do not explain

Why marketers cluster

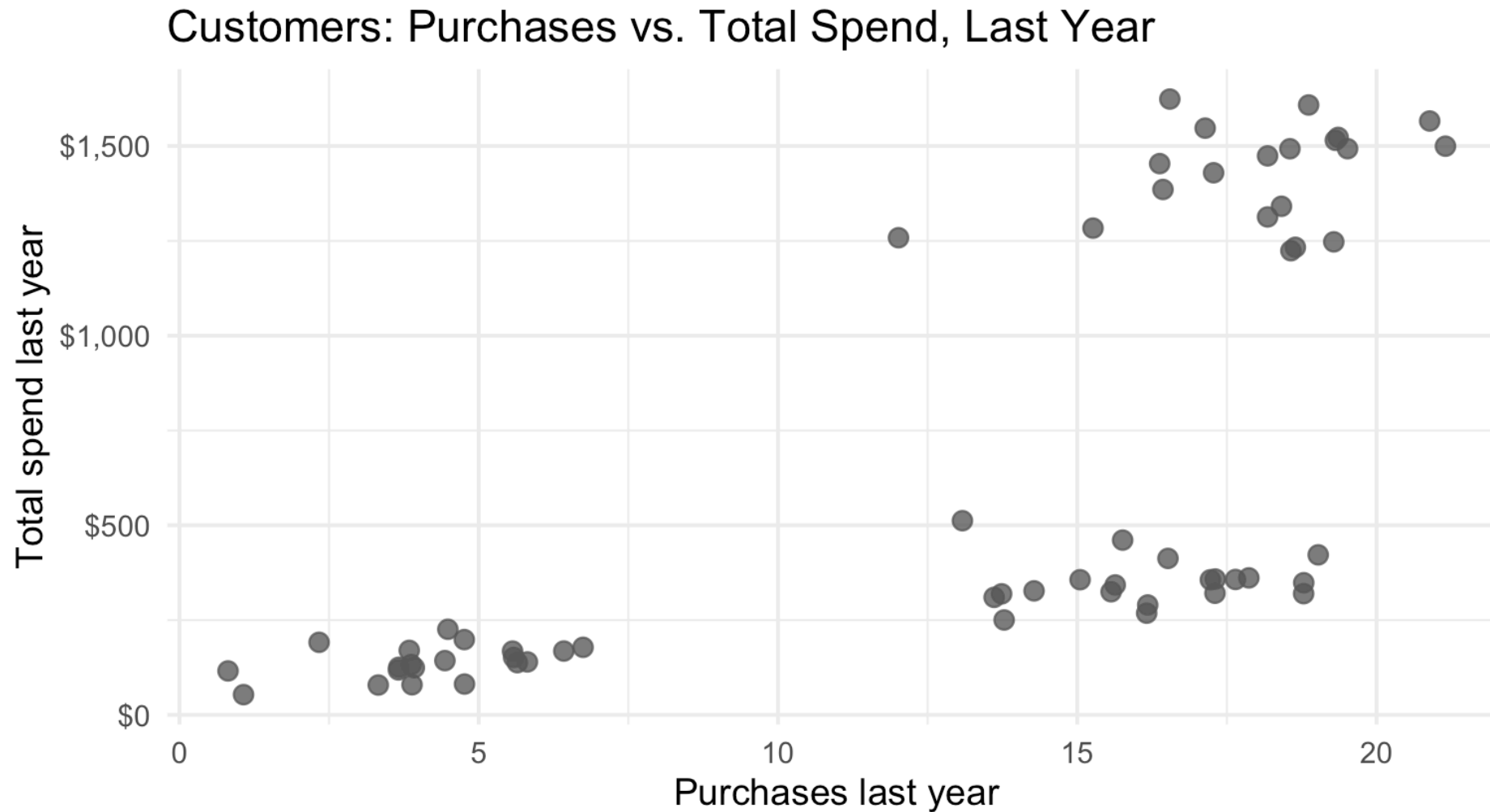
Grouping similar customers is **segmentation**, and almost everything in marketing is segment-specific:

- **Messaging**: bargain hunters get the discount email, service seekers get the concierge pitch
- **Product and pricing**: which features matter to which group, and who will pay for them
- **Budget**: which segments are growing, and where the next dollar goes

Two tools today. **K-means** finds the groups. **PCA** draws the map you need to see them, and it powers the classic marketing **perceptual map** along the way.

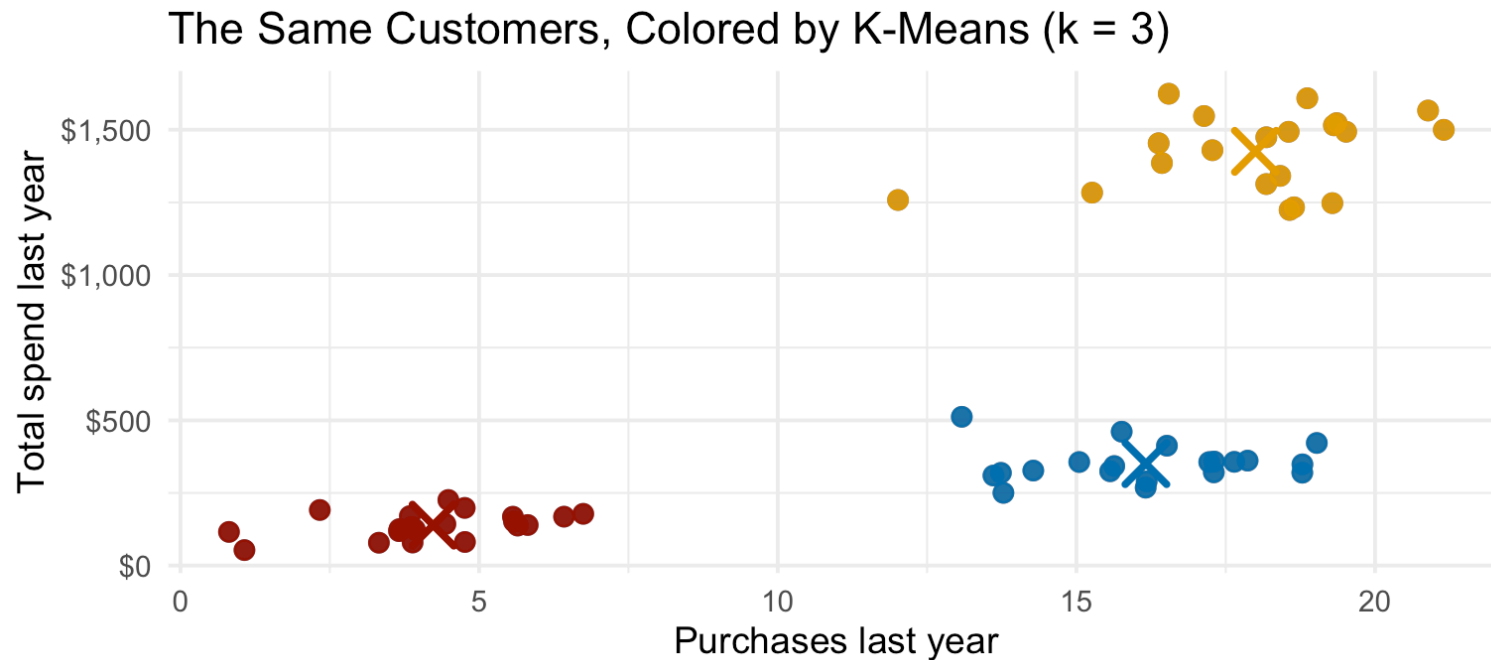
Finding Groups: K-Means

Sixty customers, two numbers



Each dot is a customer: how often they bought, and how much they spent in total. How many groups do you see, and how would you describe each one?

The same chart, after k-means



- We told k-means only two things: the **numbers** and **k = 3**, the number of groups to find. It found the same groups your eyes did
- The **X** marks each cluster's center. The groups have obvious names: occasional shoppers, frequent small-basket shoppers, big spenders
- Eyes work with 2 columns and 60 customers. With **6 columns and thousands of customers** they do not. The algorithm scales; the logic stays the same

How k-means works

You pick **k**, the number of clusters. The algorithm does the rest:

1. Place k **centers** at random
 2. **Assign** every customer to the nearest center
 3. **Move** each center to the middle of the customers assigned to it
 4. Repeat 2 and 3 until the centers stop moving
- “Nearest” means plain **distance** between rows of numbers, so every column you feed it counts
 - The random start means results can vary run to run: `set.seed()` makes them reproducible, and `nstart = 20` runs it 20 times and keeps the best

K-means never sees names, labels, or meanings. It groups rows of numbers by distance. Everything the clusters *mean* comes later, from you.

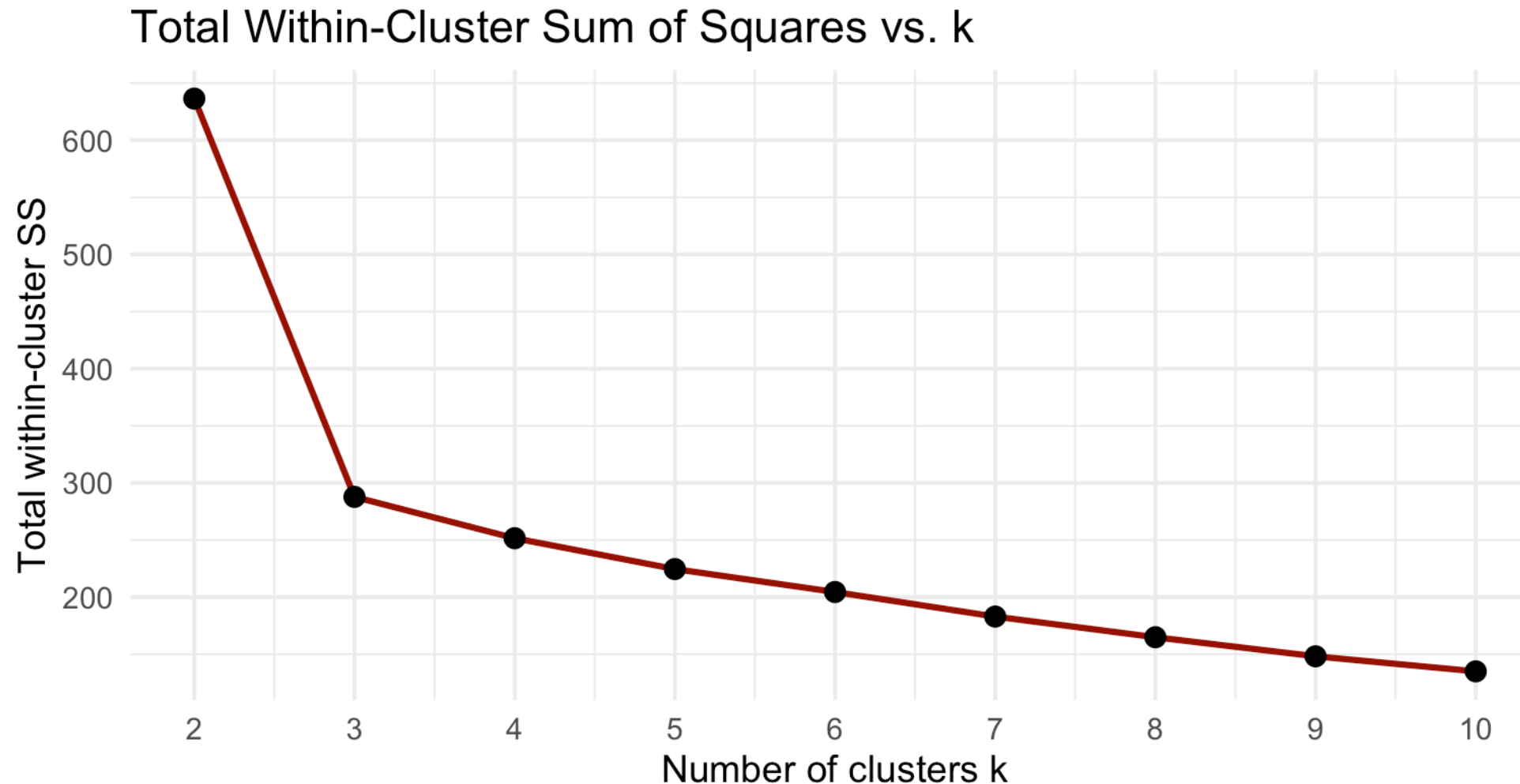
Watch the units: distance and scale

Suppose you cluster on **income** (\$15,000 to \$70,000) and **age** (20 to 60):

- Two customers \$1,000 apart in income are 1,000 “units” apart; 25 years apart in age, only 25. The distance, and so the clusters, would be **all income and no age**
- The fix: **standardize** first, replacing each column with z-scores (subtract the mean, divide by the standard deviation). In R: `scale(data)`. Then every column counts equally

Standardize whenever the columns have different units. Today’s survey ratings all share the same 1–10 scale, so we can skip it, and keep the cluster averages in interpretable 1–10 units.

How many clusters? The elbow curve



For each k, run k-means and add up the squared distances from every point to its cluster center: the “spread left inside the clusters”. Where does the curve bend?

The elbow: reading it



- Going from 2 to 3 clusters cuts the within-cluster spread from **636 to 288**. From 3 to 4 saves only 36 more, and the curve flattens: the **elbow is at k = 3**
- More clusters always fit better, so “best fit” cannot pick k: with one cluster per customer the fit is perfect and you have learned nothing
- The elbow is a guide, not a law: k must also make **business sense** (three campaigns, not nine)

Segmenting Office-Store Shoppers

The data: an importance survey

40 respondents rated how important six store attributes are to them when buying office equipment, each on a **1–10** scale:

`variety_of_choice`, `electronics`, `furniture`, `quality_of_service`, `low_prices`, `return_policy`

- We also know three things about each respondent: **professional** (0/1), **income** (in \$1,000s), and **age**
- We cluster on the **six ratings only**. The three descriptors stay out, and describe the segments afterwards
- The file is Excel, so a new reader: `read_excel()` from the **readxl** package (`fread()` is for CSVs)

Data from *R for Marketing Students*. Script: `w5-1-clustering-class.R`, data: `data/segmentation_office.xlsx`, both in `w5-code.zip`.

K-means in R: one function

```
1 dd <- as.data.table(read_excel("data/segmentation_office.xlsx", sheet = "SegmentationData"))
2 ratings <- dd[, .(variety_of_choice, electronics, furniture,
3                   quality_of_service, low_prices, return_policy)]
4 set.seed(123) # random starts, reproducible
5 km <- kmeans(ratings, centers = 3, nstart = 20) # k = 3, keep the best of 20 runs
6 km$size # respondents per cluster
```

```
[1] 14 18 8
```

```
1 dd[, cluster := as.factor(km$cluster)] # attach the labels: 1, 2, or 3
```

- Three clusters of **14, 18, and 8** respondents. Each respondent now carries a cluster label
- The labels are just 1, 2, 3, in no particular order. What they *mean* is the next slide's job

The profile table

The average rating of each attribute, within each cluster:

```
1 dd[, lapply(.SD, function(x) round(mean(x), 1)), by = cluster,  
2   .SDcols = names(ratings)][order(cluster)]
```

	cluster	variety_of_choice	electronics	furniture	quality_of_service	low_prices	return_policy
	<fctr>	<num>	<num>	<num>	<num>	<num>	<num>
1:	1	6.9	2.8	1.4	3.5	8.3	6.3
2:	2	9.1	6.1	5.8	2.4	3.7	3.2
3:	3	5.0	4.4	1.8	8.5	2.5	4.4

Read it row by row: what does each cluster rate **high**, and what does it rate **low**? Try to give each row a name before the next slide.

Naming the segments

Cluster (n)	Rates high	Rates low	The segment
1 (n = 14)	low prices 8.3 , return policy 6.3	furniture 1.4, electronics 2.8	bargain hunters
2 (n = 18)	variety 9.1 , electronics 6.1 , furniture 5.8	service 2.4, low prices 3.7	one-stop shoppers
3 (n = 8)	quality of service 8.5	low prices 2.5, furniture 1.8	service seekers

- The names come from the **table**, not from the algorithm: k-means only ever says 1, 2, 3
- Check the names against the low ratings too: one-stop shoppers do not care about price; service seekers actively dislike discount-store vibes

A segment name is a claim about the table. If a colleague cannot trace your name back to the averages, rename it.

Who is in each segment?

The three columns we kept **out** of the clustering describe who landed in each segment:

Segment	Professional	Income	Age
Bargain hunters (14)	50%	\$32,000	31
One-stop shoppers (18)	33%	\$48,000	44
Service seekers (8)	75%	\$48,000	49

- Bargain hunters are the **young, lower-income** group; service seekers are **older professionals**; one-stop shoppers sit in between with the largest headcount
- This is the actionable half: ratings say **what to offer** each segment, descriptors say **who and where they are**

Because these columns played no part in forming the clusters, the differences across segments are a finding, not something we built in.

Six Dimensions on One Screen: PCA

The problem, and the squash

The toy chart worked because there were **two** columns. Our shoppers live in **six**. A screen has two axes; you cannot plot six.

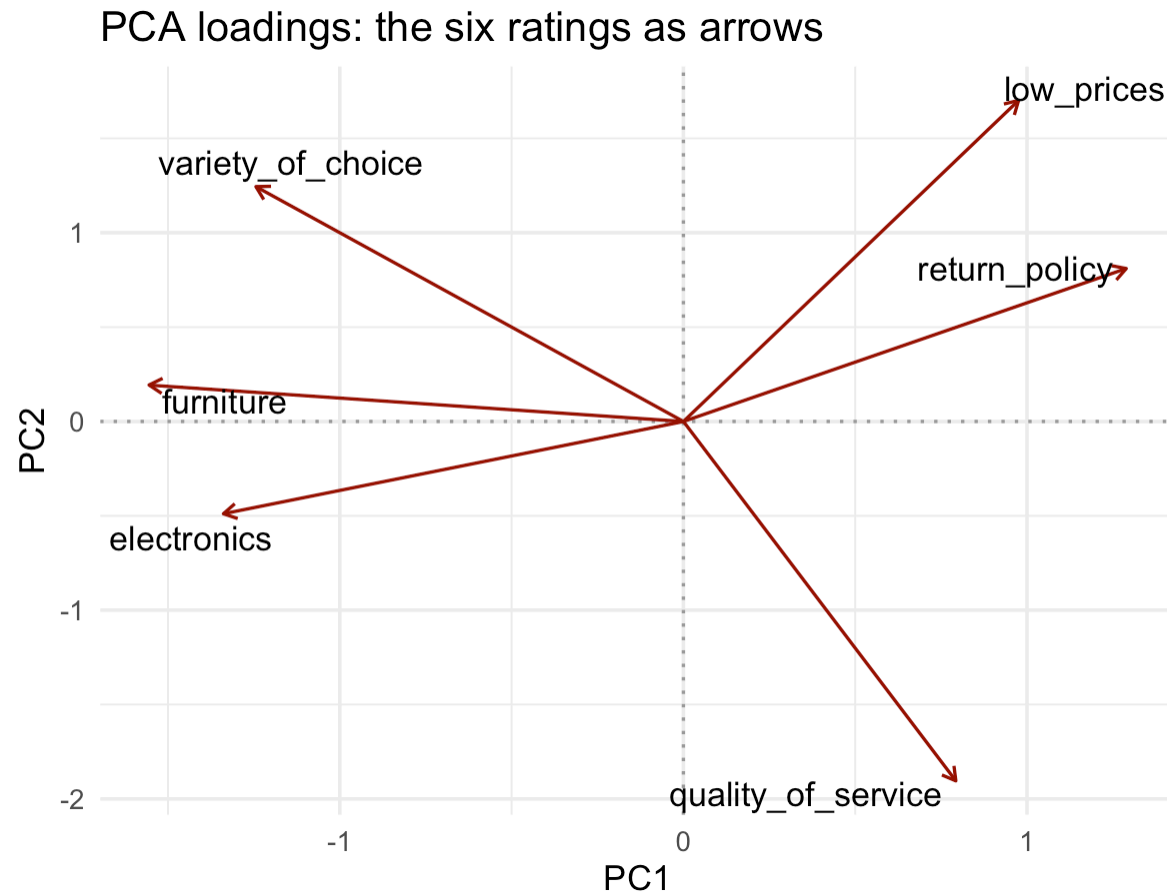
- **PCA** (principal component analysis) squashes the six columns into **two new axes**, chosen so the map keeps as much of the respondents' **variation** as possible (Tuesday we squashed a score into a probability; today we squash six columns into two axes)
- Each respondent gets a coordinate on each new axis: **PC1** and **PC2**. Plot those two and you have the best two-axis picture of the data

```
1 pca <- prcomp(ratings, center = TRUE, scale. = TRUE)
2 round(summary(pca)$importance[2, ], 2) # share of variation per component
```

PC1	PC2	PC3	PC4	PC5	PC6
0.52	0.26	0.09	0.05	0.04	0.03

- PC1 keeps **52%** of the variation, PC2 another **26%**: the 2D map shows **78%** of what six columns contain. Good enough to look at

What do the two axes mean?



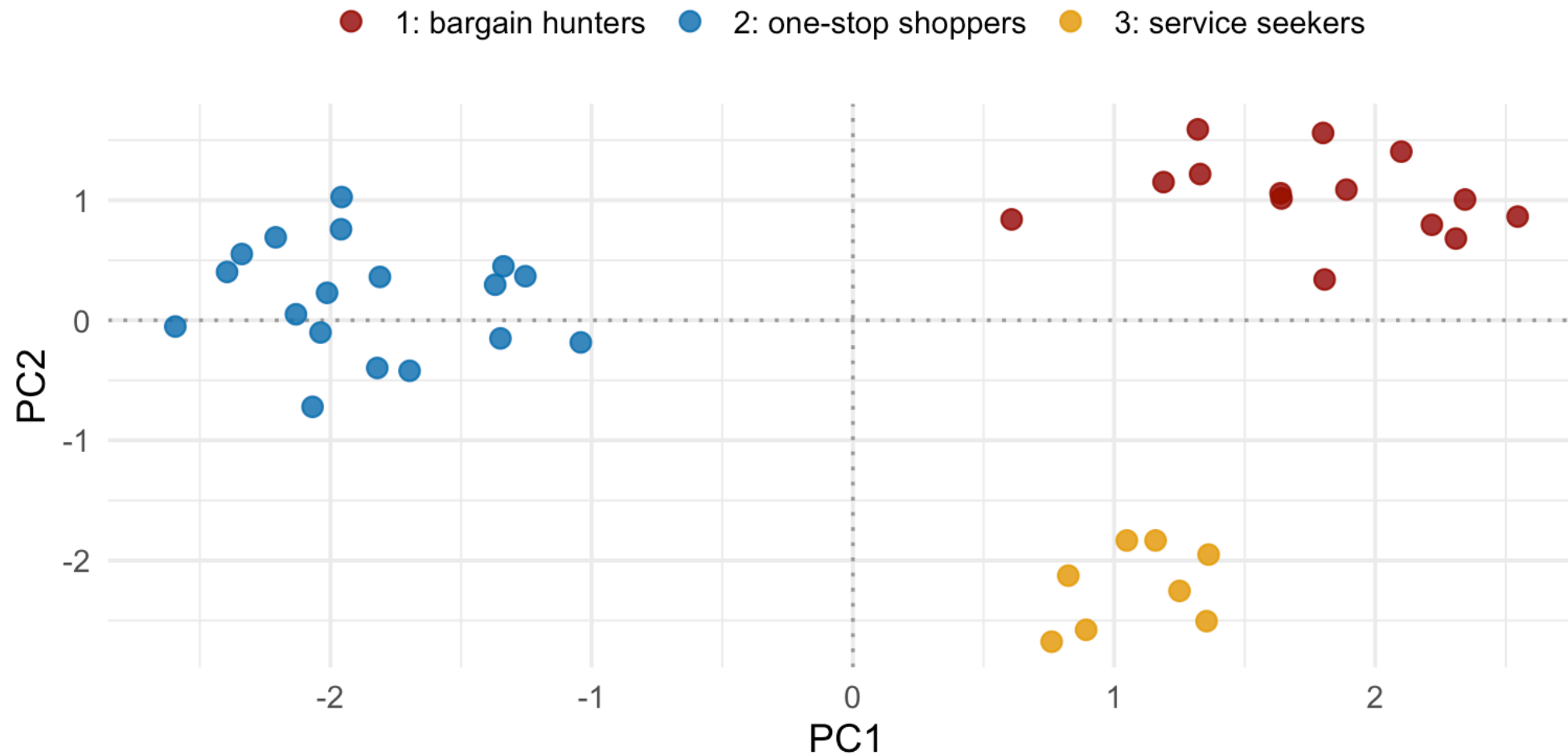
Each arrow is one of the six ratings; its direction shows where that rating **pulls** a respondent on the map:

- **Left:** variety, electronics, furniture. The left side of the map is “wants a big assortment”
- **Upper right:** low prices, return policy. **Lower right:** quality of service
- So PC1 separates *assortment* from *price and service*; PC2 separates *price* from *service*

The signs are arbitrary (R could have flipped left and right). Directions and groupings are what matter.

The map: 40 shoppers, colored by cluster

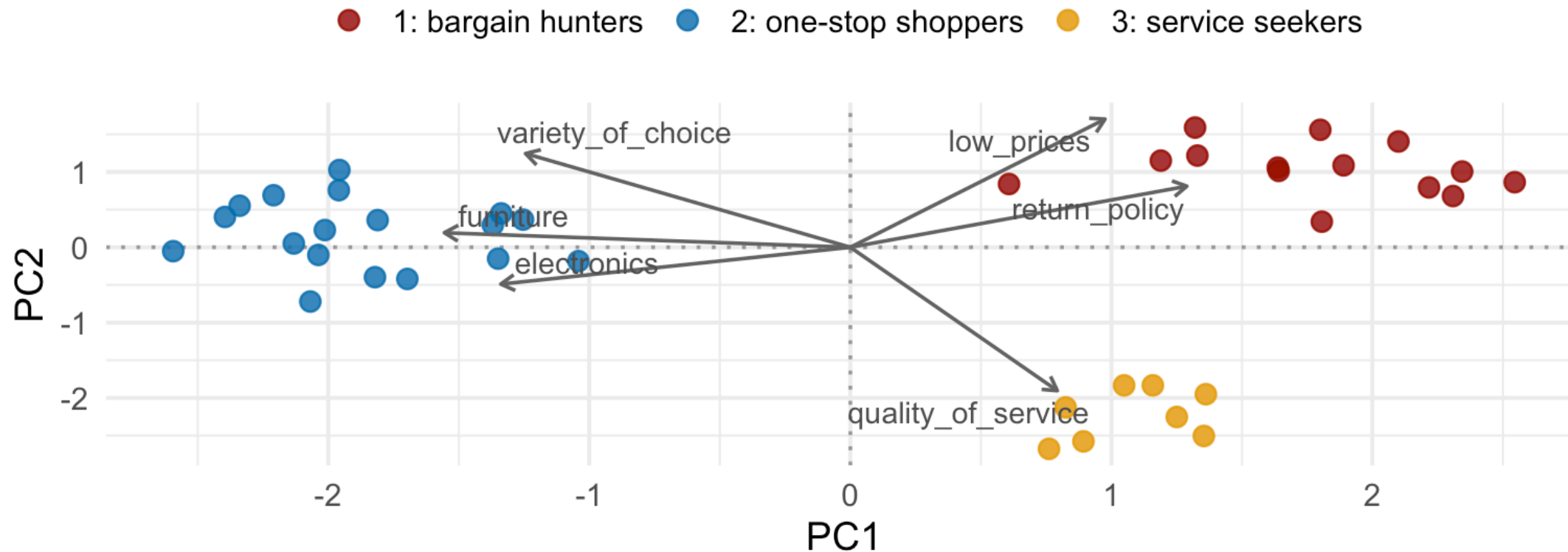
Respondents on the PCA Map, Colored by K-Means Cluster



PCA drew the map; k-means picked the colors. The two never talked to each other. Do the colors form regions?

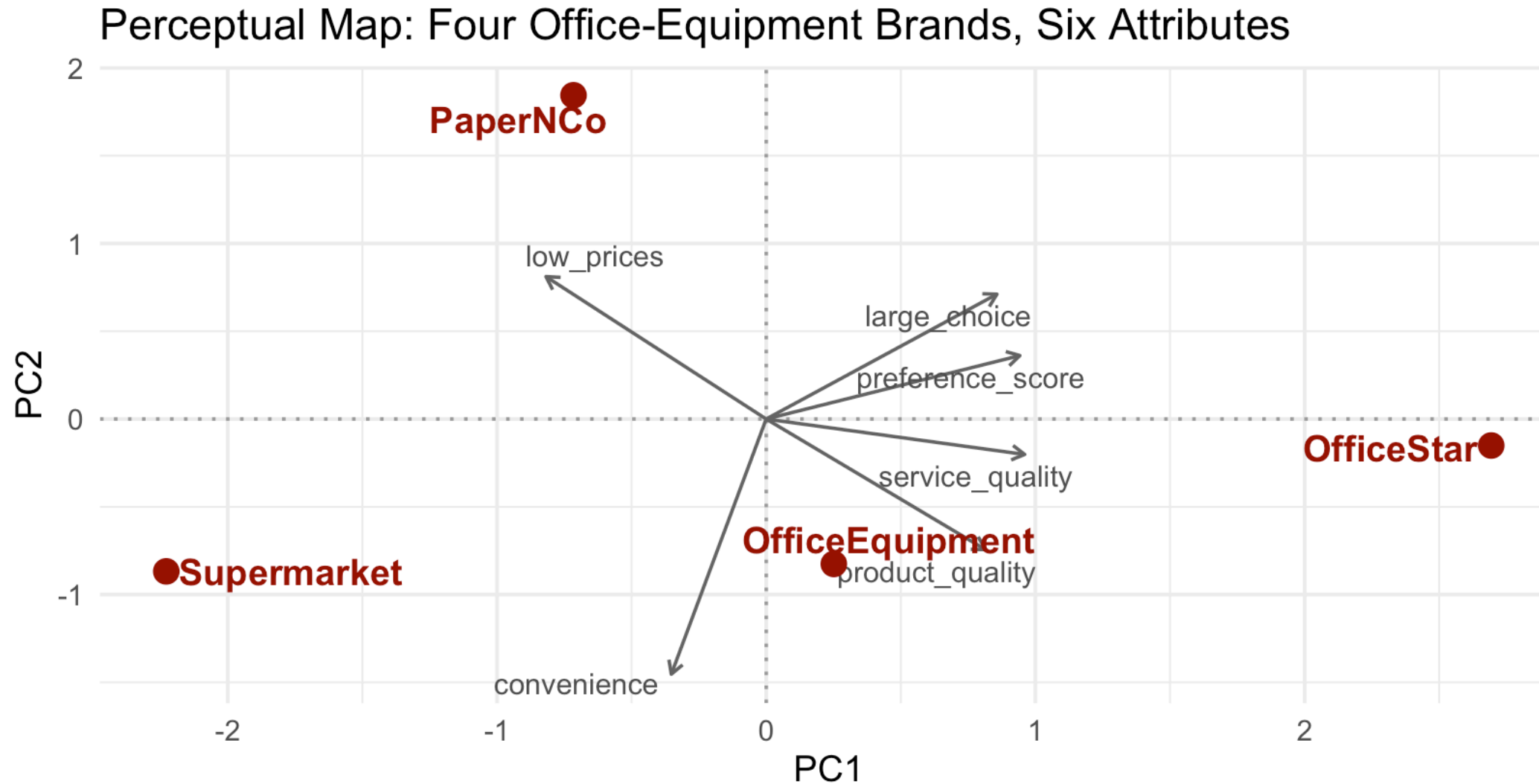
The map: reading it

The Same Map, with the Attribute Arrows (a Biplot)



- The clusters occupy **separate regions**, and each sits at the tip of the arrows its profile predicted: bargain hunters by *low prices*, one-stop shoppers by *variety and furniture*, service seekers by *quality of service*
- Two independent tools agree: the segments are in the data, not an artifact of one algorithm

The same trick on brands: a perceptual map



Same PCA, new rows: **brands** instead of shoppers, rated on six attributes. Which brands compete head-to-head? Who owns which attribute?

The perceptual map: reading it

- **OfficeStar** sits alone on the right, with *service, product quality, large choice*, and, crucially, the **preference** arrow: the strong brand people like best
- **Supermarket** is its mirror image: far left, near *low prices* and *convenience*, far from quality and preference. **PaperNCo** owns the cheap corner; **OfficeEquipment** competes on convenience with no clear strength
- Two axes hold **98%** of the variation here, so this map is essentially the whole story

Perceptual maps answer positioning questions: who are my real competitors (neighbors on the map), what do I own (my nearest arrows), and where is the **gap** nobody occupies. Script: [w5-1-pca-class.R](#).

Clustering and PCA, side by side

	K-means	PCA
Question	who is similar to whom?	which directions matter?
Output	a group label per row (1, 2, 3)	two new columns (PC1, PC2) per row
You choose	the columns, and k	the columns
Read it via	the profile table	the loadings (arrows)
Marketing use	segmentation	perceptual maps, visualization
Together	PCA draws the map, k-means draws the borders	

Cautions that apply to both: standardize when units differ; the labels and axes mean nothing until you read them off a table; and clusters **describe** customers, they do not explain them. “Service seekers are older” is a pattern, not a mechanism.

Wrap-up

Code and data to reproduce everything, on the course website:

- [w5-code.zip](#): both scripts and both datasets
- [w5-1-clustering-class.R](#): elbow, k-means, profiles, the PCA map
- [w5-1-pca-class.R](#): the brand perceptual map

Required reading: chapters [5](#), [6](#), and [7](#) of *R for Marketing Students*. Optional: chapters 11.1–11.3 of *R for Marketing Research and Analytics*. Tuesday: recommendation systems.

Questions?